



Team Omega

Audit of Backed Token Upgrade Final Report

May 13, 2026

Summary	2
Disclaimer	4
Severity definitions	4
Findings	4

General	4
G1. Test coverage is not complete [low] [resolved]	4
BackedAutoFeeToken.sol	5
BAFT1. BackedAutoFeeToken should inherit from IBackedAutoFeeToken [low] [resolved]	5
WrappedBackedTokenImplementation.sol	5
WBT1. A sanctioned Depositor Can Bypass Sanctions Check [medium] [resolved]	5
WBT1. The contract does not follow ERC4626 standard [medium][not resolved]	7
WBT2. Deposit event does not report correct values [low] [not resolved]	9
WBT3. IBackedTokenImplementation does not guarantee compatibility with WrappedBackedTokenImplementation [info] [resolved]	9
WBT4. Initialization in constructor [info] [resolved]	10

Summary

This document is the result of an audit by Team Omega for Backed Finance of an update of their Atomic Swap contract suite with a white list.

We found no high severity issue - these are issues that can lead to a loss of funds, and are essential to fix. We classified 2 issues as “medium” - these are issues we believe you should definitely address. We did classify 3 issues as “low”, and an additional 2 issues were classified as “info” - we believe the code would improve if these issues were addressed as well.

Severity	Number of issues	Number of resolved issues
High	0	0
Medium	2	1
Low	3	2
Info	2	2

Scope of the Audit

The audit concerns an upgrade of the Backed Token Contract, adding an ERC4626 wrapper for the BackedAutoFeeToken. specifically the following PR:

<https://github.com/backed-fi/backed-token-contract/pull/48>

Specifically, this PR seeks to merge commit `08272b36bd9bb36ac7c63806f9a3e23a54ee1f9a` into the current main branch at commit `040534acd89a87692b33a21c95ebff788282cf8a`

This PR adds around 100 new lines of code (not counting blank lines or comments), and removes around 40 lines.

We expect a second iteration of an update of this code of roughly the same size - the current quote includes the audit of these changes, as long as the size of these changes does not exceed the size of the current PR significantly (i.e. is in the range of 100 new lines of code).

We audited a previous version of this code before - that audit report is here:

<https://github.com/OmegaAudits/audits/blob/main/202510-Backed-rebasing-tokens.pdf>

Team Omega

Team Omega is a small team of auditors has many years of experience in developing smart contracts on Ethereum, and has a pluri-annual experience auditing Solidity code

Liability

The audit will be on a best-effort basis. In particular, the audit does not represent any guarantee that the software is free of bugs or other issues, nor is it an endorsement by Team Omega of any of the functionality implemented by the software.

Methods Used

The contracts were compiled, deployed, and tested in a test environment. We ran static analysis tools and AI tools.

Code Review

We manually inspected the source code to identify potential security flaws. We also inspected the context in which the contracts were developed, namely the Chainlink Bridge architecture.

Automatic analysis

We have used static analysis tools to detect common potential vulnerabilities. No high severity issues were identified with the automated processes. Some low severity issues were found, and we have included them below in the appropriate parts of the report.

Resolution

After we delivered a preliminary report, most of the issues were addressed by the team in the following commit:

`0fa75247631964ca4f49f54c46d874d9c68367e2`

We checked for each of the issues in our report if the issue was addressed (or not) and marked our observations below. We also audited the other changes that were made in this commit.

Disclaimer

The audit makes no statements or warranties about utility of the code, safety of the code, suitability of the business model, regulatory regime for the business model, or any other statements about fitness of the contracts to purpose, or their bug free status. The audit documentation is for discussion purposes only.

Severity definitions

High	Vulnerabilities that can lead to loss of assets or data manipulations.
Medium	Vulnerabilities that are essential to fix, but that do not lead to assets loss or data manipulations
Low	Issues that do not represent direct exploit, such as poor implementations, deviations from best practice, high gas costs, etc
Info	Matters of opinion

Findings

General

G1. Test coverage is not complete [low] [resolved]

Test coverage is not complete, and the new `WrappedBackedTokenFactory` is not tested at all:

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Lines
contracts/	93.36	89.5	96.63	93.1	
BackedAutoFeeTokenFactory.sol	100	100	100	100	
BackedAutoFeeTokenImplementation.sol	100	87.5	100	97.71	195,196,197
BackedFactory.sol	100	100	100	100	
BackedTokenImplementation.sol	100	100	100	100	
ERC20PermitDelegateTransfer.sol	100	100	100	100	
SanctionsList.sol	100	100	100	100	
WrappedBackedTokenFactory.sol	0	0	25	0	... 150,155,159
WrappedBackedTokenImplementation.sol	100	100	100	100	
contracts/interfaces/	100	100	100	100	
IBackedAutoFeeToken.sol	100	100	100	100	
contracts/oracle/	100	97.62	100	100	
BackedOracle.sol	100	96.88	100	100	
BackedOracleFactory.sol	100	100	100	100	
BackedOracleForwarder.sol	100	100	100	100	
BackedOracleInterface.sol	100	100	100	100	
All files	94.66	90.91	97.46	94.38	

Severity: Low

Resolution: Test coverage is now at 100% for all files in scope

BackedAutoFeeToken.sol

BAFT1. BackedAutoFeeToken should inherit from IBackedAutoFeeToken [low] [resolved]

To make sure that the `BackedAutoFeeToken` implementation actually implements the `IBackedAutoFeeToken` interface, have it inherit from it. This will also reduce the possibility of implementing breaking changes in later iterations.

Recommendation: Have `BackedAutoFeeToken` inherit from `IBackedAutoFeeToken`

Severity: Low

Resolution: The issue was resolved as recommended

WrappedBackedTokenImplementation.sol

WBT1. A sanctioned Depositor Can Bypass Sanctions Check [medium] [resolved]

In `_deposit`, the call chain is `_mint(receiver, shares) → _beforeTokenTransfer(address(0), receiver, shares)`. Since `from == address(0)` during minting, only the `receiver` is sanctions-checked — not the caller (depositor).

This means that sanctioned user can call `deposit/assets, nonSanctionedAddress)` to transfer their funds through the vault into a clean address without being blocked. An important caveat here is

that if the caller is sanctioned in the underlying contract, the call *will* revert. So as long as the underlying contract uses the same sanctionsList as the wrapper contract, the issue does not occur.

A related observation is that `delegatedTransfer` has no sanction check on the submitter of the transaction.

Recommendation: There are different ways to address the issue. You could add a sanctions check for the depositor. Or use the `sanctionsList` from the underlying contract instead of configuring a separate `sanctionsList` for the wrapper contract. Or, if the `sanctionsList` is only relevant for transfer of the underlying tokens, you could move the `sanctionsList` completely from the wrapper token and rely on the checks made in the underlying token.

Resolution: The contract now uses the underlying `sanctionslist`, so having two different `sanctionlists` is not possible anymore

WBT1. The contract does not follow ERC4626 standard [medium][not resolved]

Materially, the wrapper contract is a kind of interface with an underlying `IBackedAutoFeeToken`. This underlying contract itself manages “shares” that represent a number of underlying “assets”, and that number may change over time.

The ERC4626 functions `mint`, `deposit`, `withdraw`, and `redeem`, exchange shares from the underlying `IBackedAutoFeeToken` to native shares defined by the wrapper token, on a 1-1 basis. I.e. if a user calls `mint(amountShares, receiver)`, the contract will transfer `amountShares` from the underlying token from their account, and create `amountShares` of new ERC4626 shares and transfer these to the receiver. This is straightforward.

However, much of the interface as it is implemented introduces values that refer to a third token - to the *asset* amounts of the underlying `IBackedAutoFeeToken`. For example, although a call to `mint(amountShares, receiver)` simply exchanges shares for shares on a 1-1 basis, the return value of the call, and the corresponding `Deposit` event that is emitted, refer to the amount of underlying assets represented by those shares.

This mix of concrete behavior operating on the “shares” level of the underlying token, but, sometimes, using the underlying assets of the underlying token on the API level (in function calls such as `deposit`, in return values, or in Events) is confusing, and is not following the ERC4626 standard (<https://eips.ethereum.org/EIPS/eip-4626>). It will break standard integrations with ERC-4626 tokens.

Recommendation: It depends very much on your use case what the preferred behavior is here. If integration with third parties is not a priority, perhaps deviating from the ERC-4626 expectations is not a problem - although any users of this contract will have to be made aware that this contract does not work as could be expected. If, instead, the intent is to implement a strictly ERC-4626 compliant interface for the `IBackedAutoFeeToken`, it probably makes more sense to have the contract’s “asset” and “shares” to correspond exactly with the “assets” and “shares” of the underlying `IBackedAutoFeeToken`.

Severity: Medium

Resolution: As the underlying tokens tracks balances in the form of shares instead of the underlying assets, it is impossible to implement the kind of precision that the ERC4626 standard requires. So this issue presents an inherent limit in the implementation of the BackedAutoFeeToken. The deviation from the ERC4626 standard is minimal - there are rounding errors, so are on the level of “wei”, and so no significant amount of tokens are lost.

WBT2. Deposit event does not report correct values [low] [not resolved]

A call to `deposit(assetAmount, receiver)` will calculate how many shares correspond to the `assetAmount` of assets from the underlying token (using `previewDeposit` and rounding down), resulting in `sharesAmount`. It will subsequently reconvert `sharesAmount` of shares to an estimate of the underlying tokens (using `convertToAssets`, and therefore rounding down again), resulting in `assetAmount2`. It then emits an event `Deposit(caller, receiver, assetAmount2, sharesAmount)`

The problem here is that `assetAmount2` can be less than `assetAmount`. This is unexpected.

A similar issue exists in the withdraw logic.

Recommendation: To fix this local bug, use the original `assetAmount` when emitting the event, i.e. replace:

```
uint256 assets = convertToAssets(shares);  
emit Deposit(caller, receiver, assets, shares);
```

with

```
emit Deposit(caller, receiver, assetsRequested, shares);
```

And similarly, in `_withdraw`, use:

```
emit Withdraw(caller, receiver, owner, assetsRequested, shares);
```

However, also consider addressing WBT1.

Resolution: The issue was not resolved. Referring to issue WBT1, there is no real definitive solution here, as the reported numbers are by necessity rounded approximations.

WBT3. IBackedTokenImplementation does not guarantee compatibility with WrappedBackedTokenImplementation [info] [resolved]

The ERC4626 initialization code calls the “decimals” function of the underlying contract, and falls back to 18 if that function is not implemented.

Recommendation: To line up expectations, add the `decimals` function to `IBackedTokenImplementation`, either by explicitly defining it, or by inheriting from `IERC20`

WBT4. Initialization in constructor [info] [resolved]

The implementation contract is currently locked by calling `initialize` with dummy values -in the constructor. This works, but the preferred method in Open Zeppelin for locking the implementation is by using `_disableInitializers` in the constructor

Recommendation: Use `_disableInitializers` to disable the initialization logic